DQX checks: reference with examples

This is a general reference for the checks in DQX (`databricks-labs-dqx` , version 0.16.0). It is not specific to one pipeline. For each check it gives what it does, when to use it, its arguments, and an example.

The examples are written in the YAML format DQX reads. Each entry is a check with a `function` and its `arguments` . The examples were checked against the v0.16.0 source: the function names and argument names are the real ones. Test any example on your own data before you rely on it.

Contents

1. How checks are written
2. Null and empty
3. Allowed values
4. Numbers and ranges
5. Dates and times
6. Text formats
7. Standard codes
8. Network and JSON
9. SQL checks
10. Dataset-level: uniqueness and keys
11. Dataset-level: totals and aggregates
12. Dataset-level: outliers
13. Dataset-level: freshness and gaps
14. Dataset-level: schema
15. Dataset-level: comparing tables

---

1. How checks are written

A check is one entry in a list:

```
checks:
  - name: customer_id_not_null          # optional, but makes results readable
    criticality: error                  # error (default) or warn
    filter: "region = 'EU'"             # optional: only check rows matching this
    check:
      function: is_not_null             # the DQX check to run
      arguments:
        column: customer_id             # the check's own arguments
```

- **`name`** labels the result. If you leave it out, DQX builds one.

- `criticality` :
- `error` : a failing row is marked as an error. The default.
- `warn` : a failing row is marked as a warning.
- In DQX, "error" and "warn" only describe the result. What happens to failing rows depends on how you apply the checks. Our pipeline treats every failure the same way, so we don't use `warn` .
- `filter` limits the check to matching rows. Rows that don't match are not checked.
- `for_each_column` applies one check to several columns:

```
- criticality: error
  check:
    function: is_not_null_and_not_empty
    for_each_column: [sensor_id, machine_id]
```

- `user_metadata` adds your own labels to each failure record, such as a category or an owner. Use it to group failures in reports.

Python can use the same checks with `DQRowRule` and `DQDatasetRule` . The YAML form is the one used in our rules files.

Quoting literal values. In list arguments like `allowed` , a bare word is read as a column name. To compare with a literal, quote it: `allowed: ["'pending'", "'completed'"]` .

Null handling. Each check handles nulls differently. Most of the checks below say what happens to nulls. When it matters, use a separate `is_not_null` check next to the others.

---

2. Null and empty

These check whether a value is missing, or is an empty string.

`is_not_null`

Fails when the value is null.

Use it for: primary keys, required dates, numbers, and flags.

```
- name: order_id_not_null
  check:
    function: is_not_null
    arguments:
      column: order_id
```

`is_null`

Fails when the value is not null. Use it to say a column must stay empty.

```
- name: discount_must_be_null
  check:
    function: is_null
    arguments:
      column: legacy_discount
```

`is_not_empty`

Fails when the value is an empty string. A null passes.

Use it for: optional text, where a blank is a mistake but a missing value is fine.

```
- name: nickname_not_empty
  check:
    function: is_not_empty
    arguments:
      column: nickname
      trim_strings: true       # treat "   " as empty
```

`is_empty`

Fails when the value is not an empty string. A null passes. The opposite of `is_not_empty`.

```
- name: placeholder_is_empty
  check:
    function: is_empty
    arguments:
      column: placeholder_text
```

`is_not_null_and_not_empty`

Fails when the value is null or an empty string.

Use it for: required text, such as names, codes and IDs.

```
- name: customer_name_required
  check:
    function: is_not_null_and_not_empty
    arguments:
      column: customer_name
      trim_strings: true
```

`is_null_or_empty`

Fails when the value is not null and not empty. This is the opposite of `is_not_null_and_not_empty`. It is rarely what you want, but it is useful to flag rows where a column is filled in.

```
- name: legacy_code_must_be_blank
  check:
    function: is_null_or_empty
    arguments:
      column: legacy_code
```

`is_not_null_and_not_empty_array`

Fails when an array column is null or has no items.

Use it for: list columns, such as tags or line items.

```
- name: tags_present
  check:
    function: is_not_null_and_not_empty_array
    arguments:
      column: tags
```

---

3. Allowed values

`is_in_list`

Fails when the value is not in the list. A null passes.

Use it for: status codes, categories, and types, where a null is acceptable.

```
- name: status_allowed
  check:
    function: is_in_list
    arguments:
      column: status
      allowed: ["'pending'", "'completed'", "'refunded'"]
      case_sensitive: true
```

`is_not_in_list`

Fails when the value is in the forbidden list. A null passes.

Use it for: blocking known bad values.

```
- name: no_test_accounts
  check:
    function: is_not_in_list
    arguments:
      column: account_type
      forbidden: ["'TEST'", "'DEMO'"]
```

`is_not_null_and_is_in_list`

Fails when the value is null, or is not in the list.

Use it for: required code columns. This is the safer choice when a null should fail.

```
- name: currency_required_and_allowed
  check:
    function: is_not_null_and_is_in_list
    arguments:
      column: currency
      allowed: ["'USD'", "'EUR'", "'GBP'"]
```

`has_valid_string_case`

Fails when the text is not in the requested letter case. The allowed cases are `upper`, `lower`, `title`, and `sentence`.

Use it for: codes that must be upper case, such as currency codes.

```
- name: currency_upper_case
  check:
    function: has_valid_string_case
    arguments:
      column: currency
      case: upper
```

---

4. Numbers and ranges

`sql_expression`

Passes when the SQL condition is true. This is the most flexible check: anything you can write as a condition works.

Use it for: rules that combine columns, or that no built-in check covers.

```yaml
- name: amount_positive
  check:
    function: sql_expression
    arguments:
      expression: "amount IS NOT NULL AND amount > 0"
      msg: "amount must be present and greater than 0"
      columns: [amount]
```

`negate: true` flips the result. Use it when it's easier to write the bad case:

```yaml
- name: end_before_start
  check:
    function: sql_expression
    arguments:
      expression: "end_date < start_date"
      negate: true              # fails when end_date is before start_date
      columns: [start_date, end_date]
```

`is_in_range`

Passes when the value is between two limits, including both limits.

Use it for: amounts, percentages, and scores with a known range.

```yaml
- name: percentage_in_range
  check:
    function: is_in_range
    arguments:
      column: completion_pct
      min_limit: 0
      max_limit: 100
```

`is_not_in_range`

Fails when the value is between the two limits.

Use it for: values inside a range that are not allowed.

```yaml
- name: no_zero_price
  check:
    function: is_not_in_range
    arguments:
      column: unit_price
      min_limit: 0
      max_limit: 0
```

`is_not_less_than`

Passes when the value is at least the limit.

```
- name: quantity_at_least_one
  check:
    function: is_not_less_than
    arguments:
      column: quantity
      limit: 1
```

`is_not_greater_than`

Passes when the value is at most the limit.

```
- name: discount_at_most_50
  check:
    function: is_not_greater_than
    arguments:
      column: discount_pct
      limit: 50
```

`is_equal_to`

Passes when the value equals the given value. For decimals, set a tolerance.

```
- name: fx_rate_is_one_for_usd
  check:
    function: is_equal_to
    arguments:
      column: fx_rate
      value: 1.0
      abs_tolerance: 0.0001     # allow tiny floating-point differences
```

`is_not_equal_to`

Passes when the value does not equal the given value.

```
- name: version_not_zero
  check:
    function: is_not_equal_to
    arguments:
      column: version
      value: 0
```

---

5. Dates and times

`is_valid_date`

Passes when the text can be read as a date in the given format.

Use it for: dates stored as text. Typed `DATE` columns don't need it.

```
- name: received_date_format
  check:
    function: is_valid_date
    arguments:
      column: received_date_text
      date_format: "yyyy-MM-dd"
```

`is_valid_timestamp`

Passes when the text can be read as a timestamp in the given format.

```
- name: event_time_format
  check:
    function: is_valid_timestamp
    arguments:
      column: event_time_text
      timestamp_format: "yyyy-MM-dd HH:mm:ss"
```

`is_not_in_future`

Passes when the timestamp is not later than now plus an offset, in seconds.

Use it for: trade dates, created dates, and event times, where a future value is an error.

```
- name: trade_date_not_future
  check:
    function: is_not_in_future
    arguments:
      column: trade_date
      offset: 0          # strictly "not after now"
```

`is_not_in_near_future`

Passes when the timestamp is not in the next N seconds. A value further away passes.

Use it for: events that can't be scheduled too soon.

```
- name: event_not_next_hour
  check:
    function: is_not_in_near_future
    arguments:
      column: event_time
      offset: 3600
```

`is_older_than_n_days`

Passes when the date is at least N days before today. Set `negate: true` to fail the old dates instead.

Use it for: records that should be recent, such as a status that must be updated.

```
- name: last_updated_recent
  check:
    function: is_older_than_n_days
    arguments:
      column: last_updated
      days: 365
      negate: true       # fail when last_updated is more than 365 days old
```

`is_older_than_col2_for_n_days`

Passes when `column1` is at least N days older than `column2`.

Use it for: a date that must be well before another date, such as a start date before a settlement date.

```
- name: settlement_after_trade
  check:
    function: is_older_than_col2_for_n_days
    arguments:
      column1: trade_date
      column2: settlement_date
      days: 2
      negate: true       # fail when trade_date is less than 2 days before
settlement_date
```

`is_data_fresh`

Passes when the timestamp is no older than `max_age_minutes`, measured from `base_timestamp` or from now.

Use it for: checking that a row was loaded recently.

```yaml
- name: loaded_within_a_day
  check:
    function: is_data_fresh
    arguments:
      column: loaded_at
      max_age_minutes: 1440
```

---

6. Text formats

`regex_match`

Passes when the text matches the regular expression. Set `negate: true` to fail the matches instead.

Use it for: IDs and codes with a fixed pattern.

```yaml
- name: order_id_format
  check:
    function: regex_match
    arguments:
      column: order_id
      regex: '^ORD-[0-9]{6}$'
```

`is_valid_email`

Passes when the text is a valid email address.

```yaml
- name: contact_email_valid
  check:
    function: is_valid_email
    arguments:
      column: contact_email
```

`is_valid_uuid`

Passes when the text is a valid UUID. With `strict: true`, the UUID's version and variant are also checked.

```yaml
- name: id_is_uuid
  check:
    function: is_valid_uuid
    arguments:
      column: record_uuid
      strict: true
```

`is_valid_national_id`

Passes when the text matches the national ID format of a country. `country` is an ISO 3166 alpha-2 code, and defaults to `US`.

Use it for: personal identifiers. Mark those columns as restricted.

```
- name: national_id_valid
  check:
    function: is_valid_national_id
    arguments:
      column: national_id
      country: US
```

---

7. Standard codes

These check against the official ISO code lists.

`is_valid_currency_code`

Passes when the text is an ISO 4217 currency code. `code_format` defaults to `alphabetic` (for example `USD`).

```
- name: currency_iso
  check:
    function: is_valid_currency_code
    arguments:
      column: currency
      code_format: alphabetic
      case_sensitive: true
```

`is_valid_country_code`

Passes when the text is an ISO 3166-1 country code. `code_format` defaults to `alpha-2` (for example `DE`).

```
- name: country_iso
  check:
    function: is_valid_country_code
    arguments:
      column: country_code
      code_format: alpha-2
```

`is_valid_language_code`

Passes when the text is an ISO 639 language code. `code_format` defaults to `alpha-2` (for example `en`).

```
- name: language_iso
  check:
    function: is_valid_language_code
    arguments:
      column: language_code
      code_format: alpha-2
```

`is_valid_subdivision_code`

Passes when the text is an ISO 3166-2 subdivision code (for example `US-CA`). `country_column` lets you check that the subdivision belongs to the row's country.

```
- name: subdivision_matches_country
  check:
    function: is_valid_subdivision_code
    arguments:
      column: region_code
      country_column: country_code
```

---

8. Network and JSON

`is_valid_ipv4_address`

Passes when the text is a valid IPv4 address.

```
- name: client_ip_valid
  check:
    function: is_valid_ipv4_address
    arguments:
      column: client_ip
```

`is_ipv4_address_in_cidr`

Passes when the IPv4 address is inside the CIDR block.

```
- name: office_network_only
  check:
    function: is_ipv4_address_in_cidr
    arguments:
      column: client_ip
      cidr_block: "10.0.0.0/8"
```

`is_valid_ipv6_address`

Passes when the text is a valid IPv6 address.

```
- name: client_ipv6_valid
  check:
    function: is_valid_ipv6_address
    arguments:
      column: client_ipv6
```

`is_ipv6_address_in_cidr`

Fails when the IPv6 address is invalid, or valid but outside the CIDR block. A null input is not checked.

```
- name: ipv6_in_block
  check:
    function: is_ipv6_address_in_cidr
    arguments:
      column: client_ipv6
      cidr_block: "2001:db8::/32"
```

`is_valid_json`

Passes when the text is valid JSON.

```
- name: payload_is_json
  check:
    function: is_valid_json
    arguments:
      column: payload
```

`has_json_keys`

Passes when the JSON object has the given keys. With `require_all: true`, every key must be present. With `false`, at least one must be.

```yaml
- name: payload_has_id_and_type
  check:
    function: has_json_keys
    arguments:
      column: payload
      keys: [id, type]
      require_all: true
```

`has_valid_json_schema`

Passes when the JSON text matches a schema. Use `NOT NULL` in the schema to require a field.

```yaml
- name: payload_matches_schema
  check:
    function: has_valid_json_schema
    arguments:
      column: payload
      schema: "id INT NOT NULL, name STRING"
```

---

9. SQL checks

`sql_query`

Runs your own SQL. The query must return a condition column (true means the row fails) and, if you want per-row results, the merge columns that identify each row. Refer to the input data as `{{ input_view }}`.

Use it for: rules that need a join, a subquery, or a window function, which the built-in checks can't express.

```yaml
- name: no_duplicate_amount_per_customer
  check:
    function: sql_query
    arguments:
      query: >
        SELECT order_id,
               COUNT(*) OVER (PARTITION BY customer_id, amount) > 1 AS condition
        FROM {{ input_view }}
      merge_columns: [order_id]
      condition_column: condition
      msg: "same amount repeated for this customer"
```

Without `merge_columns`, the check applies to every row at once (all pass or all fail). That is for dataset-level metrics, not row routing.

Be careful: the query runs with the job's permissions. Only use queries from trusted sources.

---

10. Dataset-level: uniqueness and keys

These compare rows with each other, so they need all the rows of the table.

`is_unique`

Flags every row whose value (or combination of values) appears more than once. Both copies are flagged.

By default, nulls are treated as different from each other (`nulls_distinct: true`), so two null keys are not duplicates.

```
- name: order_id_unique
  check:
    function: is_unique
    arguments:
      columns: [order_id]
```

A combined key:

```
- name: line_item_unique
  check:
    function: is_unique
    arguments:
      columns: [order_id, line_number]
      nulls_distinct: true
```

`foreign_key`

Passes when each value exists in another table's column. Nulls are ignored by default. With `negate: true`, the check fails when the value does exist, which is useful for a list of blocked values.

```
- name: order_customer_exists
  check:
    function: foreign_key
    arguments:
      columns: [customer_id]
      ref_columns: [customer_id]
      ref_table: sales.reference.customers
```

Use `null_safe: true` if a null should match a null in the reference table.

---

11. Dataset-level: totals and aggregates

These compute a total (count, sum, average and so on) over the whole table or per group, and compare it with a limit.

`is_aggr_not_greater_than`

Fails when the total is above the limit.

```
- name: daily_order_count_cap
  check:
    function: is_aggr_not_greater_than
    arguments:
      column: "*"
      aggr_type: count
      limit: 100000
```

Per group, with `group_by`:

```
- name: amount_per_customer_cap
  check:
    function: is_aggr_not_greater_than
    arguments:
      column: amount
      aggr_type: sum
      limit: 50000
      group_by: [customer_id]
```

`is_aggr_not_less_than`

Fails when the total is below the limit.

```
- name: at_least_one_row_per_region
  check:
    function: is_aggr_not_less_than
    arguments:
      column: "*"
      aggr_type: count
      limit: 1
      group_by: [region]
```

`is_aggr_equal`

Fails when the total does not equal the value. Set a tolerance for sums of decimals.

```
- name: control_total_matches
  check:
    function: is_aggr_equal
    arguments:
      column: amount
      aggr_type: sum
      limit: 1234567.89
      abs_tolerance: 0.01
```

`is_aggr_not_equal`

Fails when the total equals the value.

```
- name: no_empty_batch
  check:
    function: is_aggr_not_equal
    arguments:
      column: "*"
      aggr_type: count
      limit: 0
```

---

12. Dataset-level: outliers

`has_no_outliers`

Flags values that are statistical outliers in the column. DQX uses a robust statistical test. Check the source for the exact threshold before relying on it.

```
- name: amount_no_outliers
  check:
    function: has_no_outliers
    arguments:
      column: amount
```

`has_no_aggr_outliers`

Checks whether a total over time is unusually high or low. It compares each time bucket with a rolling baseline of the preceding buckets.

Use it for: daily or hourly volumes that should be stable.

```yaml
- name: daily_volume_no_spike
  check:
    function: has_no_aggr_outliers
    arguments:
      column: "*"
      time_column: order_date
      aggr_type: count
      sigma: 3.0
      lookback_num_intervals: 14
      warmup_num_intervals: 7
      time_interval: day
```

- `sigma`: how many standard deviations count as an outlier.
- `lookback_num_intervals`: how many earlier buckets form the baseline.
- `warmup_num_intervals`: how many buckets must exist before the check starts.
- `time_interval`: `minute`, `hour`, `day`, `week`, or `month`.

---

13. Dataset-level: freshness and gaps

`is_data_fresh_per_time_window`

Checks that enough records arrive in every time window, for example at least 10 records every hour.

```yaml
- name: records_every_hour
  check:
    function: is_data_fresh_per_time_window
    arguments:
      column: ingested_at
      window_minutes: 60
      min_records_per_window: 10
      lookback_windows: 24
```

`lookback_windows` limits the check to the last N windows. Without it, the whole dataset is checked.

`has_no_gaps_per_time_window`

Fails when a time window has no rows, between windows that do. It can be run per group, and `trailing_gap` also catches a missing most-recent window.

```
- name: no_missing_days_per_store
  check:
    function: has_no_gaps_per_time_window
    arguments:
      column: sale_date
      window_minutes: 1440
      group_by: [store_id]
      trailing_gap: true
```

---

14. Dataset-level: schema

`has_valid_schema`

Checks that the table has the expected columns and types. With `strict: false`, extra columns are allowed. With `strict: true`, the columns must match exactly, in the same order.

The expected schema is written as a DDL string, or taken from another table with `ref_table`.

```
- name: orders_schema
  check:
    function: has_valid_schema
    arguments:
      expected_schema: "order_id STRING, customer_id STRING, amount DOUBLE, status
STRING"
      strict: false
```

Load the schema from another table:

```
- name: orders_schema_from_source
  check:
    function: has_valid_schema
    arguments:
      ref_table: sales.silver.orders
      strict: false
```

---

15. Dataset-level: comparing tables

`aggr_matches_dataset`

Compares a total in this table with the same total in another table. It fails when they differ, and a tolerance is allowed.

Use it for: row counts or totals that must match between layers.

```
- name: row_count_matches_source
  check:
    function: aggr_matches_dataset
    arguments:
      column: "*"
      aggr_type: count
      ref_table: sales.bronze.orders
```

Compare per group, with `group_by` and `ref_group_by`:

```
- name: amount_per_region_matches
  check:
    function: aggr_matches_dataset
    arguments:
      column: amount
      aggr_type: sum
      ref_table: sales.bronze.orders
      ref_column: amount
      group_by: [region]
      rel_tolerance: 0.001
```

`compare_datasets`

Compares two tables row by row. It matches rows by key, and reports rows that are missing, extra, or changed. The details go into the result as JSON.

Use it for: reconciling a table against a copy or a source extract.

```
- name: matches_source_copy
  check:
    function: compare_datasets
    arguments:
      columns: [order_id]
      ref_columns: [order_id]
      ref_table: sales.bronze.orders
      check_missing_records: true
      exclude_columns: [loaded_at]
```

- `columns` and `ref_columns` are matched by position, so their order must line up.
- `check_missing_records: true` also finds rows missing from this table. It can return more rows than the table has.
- `exclude_columns` skips columns like timestamps that are expected to differ.

Using these checks well

- **Start with the required columns.** Not-null and not-empty checks cover most problems in practice.
- **Put the business rule next to the column it is about.** Keep one check per rule so failures are easy to read.
- **Use dataset-level checks in separate runs.** They need the whole table, not one batch of changes.
- **Test every rule on sample data.** Run the file through DQX's `validate_checks` first. It catches a wrong function or argument name without running the job.
- **Mind the nulls.** Read each check's null behaviour above, and add an `is_not_null` check when a null should fail.

Source

The function and argument names come from `databricks/labs/dqx/check_funcs.py` in the DQX repository, tag `v0.16.0`.